qala

Solution Factory Operating System

Solution System Design Document

Solution Domain Architecture & Specification

Version 2.0 · March 2026

CONFIDENTIAL · INTERNAL USE ONLY · DRAFT

SECTION 1

Executive Summary

qala is a Solution Factory Operating System (SFOS) — a unified, hierarchical platform for designing, building, managing, and operating solutions of every type, at any scale. The Solution System is the primary domain of the platform, and the Solution is the central root element around which every subsystem, environment, factory, and workflow is organized.

Every deliverable managed in qala — whether a software application, a physical good, a market-facing product, a delivered service, or a complex platform — is represented as a typed Solution with a consistent structure, governed lifecycle, and traceable value chain. From a solo developer organizing a personal project to a global enterprise managing thousands of products and services, qala provides a single standardized interface and operating model.

qala is itself a solution factory — the root factory — capable of producing tiered, hierarchical solution factories, development environments, and solution models.

> **Core Principle:**
> *Every solution, regardless of type or scale, is created, maintained, and operated within a standardized, reproducible, and governed environment. The Solution is the root element of the qala model — everything else exists to enable the Solution.*

SECTION 2

Solution System Overview

The Solution System is the top-level domain of the qala platform. It defines the ontology for what a Solution is, how it is structured, how it is classified, and how it relates to every other system in the platform. All factories, environments, tooling, pipelines, and governance frameworks exist to serve the Solution System.

2.1 The Solution as Root Element

The Solution is the central, root element of the qala platform. Every object managed in qala either is a Solution, belongs to a Solution, or exists to support a Solution. The platform's information architecture radiates outward from this root.

Domain	Relationship to Solution	Description
Solution Component	Part-of Solution	A constituent element of a Solution; has design, parts, materials, and IDs
Solution Model	Blueprint of Solution	The design, blueprint, and prototype specification that defines a Solution
Solution Testbed	Validates Solution	The environment and test suites used to verify a Solution
Solution Factory	Produces Solution	The coordinated SDE network that manufactures Solutions
Solution Vendor	Supplies to Solution	External or internal providers of parts, components, or services
Solution Orchestration	Executes for Solution	Workflows and tasks that drive Solution creation and operation
Solution Data	Describes Solution	Metadata, features, versioning, and maturity classification
Solution Tooling	Enables Solution	Toolkits, toolsets, toolchains, and tools used to build Solutions

2.2 Solution Types

qala recognizes six canonical Solution types. Every artifact managed in the platform is classified as one of these types:

Solution Type	Description
Application	A software application with defined processes and user interactions

System	A composition of applications with coordinated behavior and shared infrastructure
Good	A tangible or digital deliverable produced as an output of a process
Product	A market-facing offering combining software, services, and/or goods
Service	A capability or function delivered to a consumer on-demand or continuously
Platform	A substrate on which other Solutions are built, deployed, and operated

SECTION 3

Solution

A Solution is the fundamental unit of value in qala. It represents a realized answer to a problem, a goal, or an objective. Solutions are typed, versioned, composed of components, governed by lifecycle states, and linked to the factories, environments, and value chains that produced them.

3.1 Solution Data

Every Solution carries a canonical metadata record and a features list. This data persists in the Solution Registry and is versioned alongside the Solution itself.

Solution Metadata

Metadata Field	Description
Unique ID	Globally unique identifier (UUID v4) assigned at Solution creation
Name	Human-readable Solution name
Version	Semantic version string (MAJOR.MINOR.PATCH)
Maturity	Lifecycle maturity level — see Maturity Model below
Solution Type	One of: Application, System, Good, Product, Service, Platform
Owner	User or organization responsible for the Solution
SDE Reference	Pointer to the SDE in which this Solution was created
Factory Reference	Pointer to the Solution Factory that produced this Solution
Created At	ISO 8601 timestamp of creation
Updated At	ISO 8601 timestamp of last modification
Tags	Freeform labels for search, discovery, and classification

Solution Maturity Model

Each Solution progresses through a defined maturity lifecycle. Promotion between stages requires passing configured gate criteria.

Maturity Stage	Code	Description	Gate Criteria
Sandbox	SANDBOX	Exploratory, unconstrained development. No stability guarantees.	None — open creation

Development	DEV	Active feature development. Nightly builds may break.	Basic build success
Nightly	NIGHTLY	Automated nightly build and test cycle. Stability improving.	CI green on main branch
Test	TEST	Feature-complete. Full test suites running. Pre-release hardening.	All test suites passing
Control Managed	CM	Release-qualified. Under Configuration Management control. Immutable.	CM board approval + sign-off

Solution Features List

A Solution's capabilities are described by a structured features list. Each feature entry carries a name, a brief summary, and a full feature specification.

Feature Field	Type	Description
Name	string	Short identifier for the feature
Brief	string	One-sentence summary of what the feature does
Feature	text	Full specification of the feature: behavior, acceptance criteria, constraints

SECTION 4

Solution Component

A Solution Component is a constituent part of a Solution. Components are the first-level decomposition of a Solution and carry design, manufacturing, and material information. A single Solution may contain many Components. Components themselves can be assembled from lower-level Parts.

This model is type-agnostic: a Component may represent a software module, a physical sub-assembly, a service capability, or any other discrete unit of the Solution. The same schema applies universally.

4.1 Component Structure

Component Field	Description
Component ID	Unique identifier for this component within the Solution
Component Name	Human-readable name
Component Type	Classification (e.g., module, sub-assembly, feature, capability)
Design / Blueprint	Reference to the design specification or blueprint document
Version	Component-level version string
Owner	Person or team responsible for this component
Solution Reference	Foreign key back to the parent Solution
Parts List	Ordered collection of Solution Parts belonging to this component

4.2 Solution Part

A Part is the lowest-level constituent of a Component. Parts carry manufacturing and material identity information sufficient to uniquely identify, source, and reproduce them.

Part Field	Description
Part ID	Unique identifier for this Part (platform-assigned UUID)
Part Number	External or internal part number (e.g., manufacturer's PN)
Part Name	Human-readable name for the Part
Part Vendor	The supplier, manufacturer, or source of this Part
Part Material	Material classification (e.g., ABS plastic, steel alloy, open-source library, SaaS API)

Part Design / Blueprint	Reference to the drawing, specification, or schema defining this Part
Part Version	Version of this Part's design or supply revision
Part Tags	Freeform labels for categorization and search

4.3 Component Examples

The following examples illustrate how the Component and Part model applies across Solution types.

Example A — Physical Good: Screwdriver

Component / Part	Part Number	Material / Notes
Handle	HDL-001	Injection-molded ABS plastic, ergonomic grip profile
Shank	SHK-001	Hardened carbon steel, hexagonal cross-section
Bit	BIT-PH2	S2 tool steel, Phillips #2 profile, heat-treated
Grip Sleeve	GRP-001	TPR rubber, over-molded onto handle

Example B — Application: Time Tracking Software

Component / Part	Part / Module ID	Description
Auth Module	MOD-AUTH-001	JWT-based authentication and RBAC
Timer Engine	MOD-TIMER-001	Core time-entry start/stop/pause logic
Reporting Module	MOD-RPT-001	Report generation, CSV/PDF export
Notifications Service	MOD-NOTIF-001	Email and in-app notification dispatch
Database Schema	PART-DB-001	PostgreSQL schema for time entries and users

Example C — Service: Business Coaching Service

Component / Part	Part ID	Description
Onboarding Package	SVC-ONBRD-001	Discovery session, intake form, goal-setting framework
Coaching Session	SVC-SESS-001	1:1 structured 60-minute coaching engagement
Playbook Library	SVC-PLAY-001	Collection of frameworks, templates, and worksheets

Progress Tracker	SVC-PROG-001	Metrics and milestone tracking dashboard
Resource Pack	SVC-RSRC-001	Reading lists, assessments, and reference material

SECTION 5

Solution Model

The Solution Model defines the formal specification for a Solution before and during its construction. It is the intellectual blueprint layer — containing designs, blueprints, and prototypes — that drives the manufacturing process executed by the Solution Factory.

5.1 Model Components

Model Element	Description
Blueprint	Formal structural specification: architecture, interfaces, data contracts, and dependency graph
Design	Visual and functional design artifacts: wireframes, schematics, drawings, and design system references
Prototype	A working instantiation of a subset of Solution capabilities for validation and feedback

5.2 Solution Structure Model

The qala Solution Structure Model defines a universal containment hierarchy for decomposing any Solution. This hierarchy is consistent across all Solution types, enabling uniform tooling, governance, and analysis.

System → Application → Process → Component → Interface → Message → Data Structure → Data

Layer	Contains	Message / Interface Type	Notes
System	Applications	—	Top-level organizational boundary
Application	Processes	—	Executable units with defined behavior
Process	Components	—	Logical groupings of functionality
Component	Interfaces	—	Implementation units with explicit contracts
Interface	Messages + Imports/Exports	Inbound / Outbound	Boundary layer defining data contracts
Message	Data Structures	Event (dynamic) / State (static)	Typed message payload

Data Structure	Data fields	—	Typed fields using qala data type system
Data	Primitive values	—	Leaf-level values using canonical data types

5.3 Data Type System

qala defines a canonical set of data types used across all Solution data structures. Custom types may be composed from these primitives.

`bool`	`string`	`int`	`float`	`double`	`null`	`varchar`	`char`	`date`
`array`	`tuple`	`set`	`map`	`object`	`pointer`	`custom`		

SECTION 6

Solution Testbed

The Solution Testbed is the verification and validation environment for a Solution. It defines the full battery of tests, test suites, testbed infrastructure, and quality gates that a Solution must pass before advancing through the maturity lifecycle.

6.1 Testbed Architecture

Testbed Element	Description
Test Case	The atomic unit of verification: a single scenario with defined inputs, expected outputs, and pass/fail criteria
Test Suite	A named, ordered collection of related test cases targeting a specific feature, component, or concern
Test Plan	The master document defining scope, schedule, environments, and acceptance criteria for a Solution release
Test Environment	An isolated, automatically provisioned environment matching the target deployment topology
Test Run	A single execution of a test suite against a specific Solution build, producing a dated results record
Defect	A recorded failure: linked to the test case that found it, the build it appeared in, and the release it blocks
Test Analytics	Aggregated metrics across runs: pass rates, trend analysis, defect density, MTTR

6.2 Test Types

Test Type	Scope	Automation Level
Unit Tests	Individual functions and modules	Fully automated; runs on every commit
Integration Tests	Service-to-service and module boundaries	Fully automated; runs on every build
System Tests	End-to-end Solution behavior	Automated; runs on nightly and release builds
Performance / Benchmark	Throughput, latency, scalability under load	Automated; runs on release candidates
Security / SAST / DAST	Static and dynamic vulnerability scanning	Automated; SAST on commit, DAST on builds

Acceptance Tests (UAT)	Business requirements validation	Semi-automated; human sign-off required
Prototype Tests	POC feasibility and design validation	Manual + AI-assisted feedback analysis
Regression Tests	Prevention of previously resolved defects	Fully automated; runs on every build

6.3 AI-Assisted Testing

The qala AI Agents service enhances the Testbed with the following intelligent capabilities:

- Automated test case generation from feature specifications and historical failure patterns
- AI-driven test prioritization based on risk, coverage gaps, and change impact analysis
- Predictive defect detection: flags high-risk code areas before testing begins
- Self-healing tests: automatically updates fragile tests when Solution interfaces evolve
- Prototype feedback analysis: synthesizes prototype results into actionable design recommendations

SECTION 7

Solution Factory

A Solution Factory (SF) is a coordinated, networked collection of Solution Development Environments (SDEs) that produces Solutions in a consistent, repeatable, and scalable way. The factory provides the production infrastructure — hermetic builds, CI/CD pipelines, artifact management, and governance controls — that transforms Solution Models into deployed Solutions.

7.1 Factory Structure

Factory Element	Description
Solution Factory	The top-level coordination entity; owns a collection of networked SDEs
SDE Network	The set of Solution Development Environments operated by this factory
Build System	Hermetic, reproducible build infrastructure for all Solutions produced by the factory
CI/CD System	Automated integration, testing, and deployment pipelines
Artifact Repository	Versioned storage for all build outputs, binaries, containers, and packages
Governance Layer	Policies, templates, standards, and approval workflows applied to all factory output
Factory Registry	Platform-level catalog of all Solution Factories, their SDEs, and produced Solutions

7.2 Hermetic Build Environments

All Solution builds within qala are executed in hermetic build environments — fully isolated, reproducible build contexts that guarantee consistent outputs regardless of when or where they are run.

- Fully isolated: no access to external network resources, host file system, or ambient credentials during build
- Reproducible: identical inputs (source, dependencies, environment variables) always produce byte-identical outputs
- Environment-as-code: all build environment definitions are version-controlled and immutable at build time
- Containerized: each build executes in a purpose-built, ephemeral container spun up and destroyed per run
- Dependency locking: all dependency versions are pinned and cryptographically verified before build execution

- Build attestation: every build produces a signed attestation record (SLSA provenance) linking output to inputs

7.3 CI/CD/CM System

Pipeline Stage	Trigger	Actions
Continuous Integration (CI)	Every commit / pull request	Build, unit test, SAST scan, artifact generation, code quality gate
Continuous Delivery (CD)	CI green + merge to main	Integration tests, system tests, DAST scan, staging deployment
Continuous Deployment	CD green + maturity gate	Automated promotion to target environment (canary, blue/green, or rolling)
Configuration Management (CM)	CM board approval	Immutable release tagging, audit trail, environment lock-down, sign-off

7.4 Factory Hierarchy

qala itself is the Root Solution Factory. It produces tiered, hierarchical child factories that inherit governance, tooling, and pipeline configurations from their parent:

- Root Factory (qala platform itself) — operates all child factories
  - Enterprise Factory — multi-team, multi-product factory with full governance
  - Team Factory — single-team factory scoped to a product or service line
  - Personal Factory — single-user factory for personal and hobby projects

SECTION 8

Solution Vendor

A Solution Vendor represents any external or internal entity that supplies components, parts, services, or capabilities to a Solution or Solution Factory. The Vendor Registry maintains a catalog of all approved suppliers and their associated Parts and Components.

8.1 Vendor Record

Vendor Field	Description
Vendor ID	Unique identifier for this vendor in the qala Vendor Registry
Vendor Name	Legal or trade name of the supplier
Vendor Type	Classification: Hardware Supplier, Software Library, SaaS Provider, Service Provider, Internal Team
Contact Information	Primary contact person, email, and communication channels
Parts Catalog	List of Parts and Components this vendor supplies, with part numbers and versions
Qualification Status	Approved, Provisional, Deprecated, or Blocked
SLA / Agreement Reference	Link to the governing contract, license, or SLA document
Lead Time	Typical delivery or provisioning time for this vendor's offerings

8.2 Vendor Qualification Workflow

Before a Vendor's Parts may be used in a CM-stage Solution, the vendor must complete the qualification workflow:

- Submission: vendor submits Parts catalog, documentation, and certification evidence
- Review: governance team validates compliance with quality and security standards
- Provisional approval: vendor may be used in SANDBOX through TEST stages
- Full approval: vendor is promoted to Approved status; Parts may be used in CM-stage Solutions
- Ongoing monitoring: SLA compliance, security advisories, and license changes are tracked continuously

SECTION 9

Solution Orchestration

Solution Orchestration defines how the work of creating, delivering, and operating a Solution is organized into structured, traceable workflows and tasks. Orchestration connects Solution Models to Solution Factories, coordinating the sequence and parallelism of all factory activities.

9.1 Orchestration Hierarchy

Level	Description
Solution Orchestration	The top-level coordination model for all work associated with a Solution
Workflow	A named, directed graph of Tasks representing a complete end-to-end process (e.g., Release Workflow, Onboarding Workflow)
Task	The atomic unit of orchestrated work: an action with defined inputs, outputs, assignee, and completion criteria

9.2 Workflow Types

Workflow Type	Description
Build Workflow	Executes the hermetic build pipeline for a Solution or Component
Test Workflow	Runs a defined test suite against a Solution build
Release Workflow	Orchestrates build → test → stage → approve → deploy for a Solution release
Deployment Workflow	Executes the deployment of a Solution artifact to a target environment
Rollback Workflow	Restores a previous Solution version in a target environment
Provisioning Workflow	Creates and configures a new SDE from a template
Governance Workflow	Routes a Solution through review, approval, and CM promotion gates
Data Pipeline Workflow	Executes extract, transform, and load operations for Solution data

9.3 Task Schema

Task Field	Description
Task ID	Unique identifier
Task Name	Human-readable label

Workflow Reference	Parent workflow this Task belongs to
Task Type	build | test | deploy | review | approve | notify | script | manual
Inputs	Data, artifacts, or conditions required to begin this Task
Outputs	Artifacts, state changes, or signals produced by this Task upon completion
Assignee	User, team, service, or AI agent responsible for execution
Status	pending | running | succeeded | failed | skipped | blocked
Dependencies	List of Task IDs that must complete before this Task may begin
Timeout	Maximum allowed execution time before the Task is marked as failed
Retry Policy	Number of retries and backoff strategy on failure

SECTION 10

Solution Tooling

Solution Tooling defines the full hierarchy of tools available within a Solution Development Environment, organized from individual tools up through toolsets, toolkits, and toolchains. Tooling is version-controlled, vendor-managed, and composable.

10.1 Tooling Hierarchy

Level	Description
Tool	The atomic unit: a single executable, library, or service that performs a specific function (e.g., compiler, linter, formatter)
Toolset	A named collection of related Tools serving a common purpose (e.g., "Go Build Toolset": go, gofmt, golint, gotest)
Toolkit	A curated collection of Toolsets covering a full domain of practice (e.g., "Backend Development Toolkit")
Toolchain	A linked, ordered sequence of Tools that operate in pipeline: each tool's output feeds the next (e.g., compile → link → sign → package)
Tool Suite	A governed, platform-approved set of Toolkits for a given Solution type or organizational standard

10.2 Tool Record

Tool Field	Description
Tool ID	Unique identifier in the qala Tool Registry
Tool Name	Human-readable name (e.g., rustc, docker, terraform)
Tool Version	Pinned semantic version used within this SDE
Tool Type	compiler | linter | formatter | build-system | test-runner | container | IaC | IDE | VCS | scanner | other
Vendor / Source	Tool publisher or open-source project origin
License	License type (MIT, Apache 2.0, proprietary, etc.)
Toolset Membership	Which Toolsets include this Tool
Toolchain Position	Position in any Toolchains this Tool participates in
Verification Hash	Cryptographic hash of the tool binary for supply-chain verification

10.3 Tooling Management

- All tool versions are pinned and stored in the SDE's configuration manifest
- Tool upgrades follow a promotion workflow: SANDBOX test → DEV validation → CM-approved update
- Third-party tool integrations are managed through the Vendor Registry; tools are subject to vendor qualification
- Toolchain definitions are version-controlled as code alongside Solution source
- AI agents monitor tool ecosystem for security advisories, deprecations, and recommended upgrades

SECTION 11

Solution Registry

The Solution Registry is the authoritative catalog for all Solutions, Solution Models, Vendors, Tools, and Factories managed on the qala platform. It provides discovery, lineage tracking, governance assignment, and cross-reference services across the entire platform.

11.1 Registry Catalogs

Registry Catalog	Contents
Solution Registry	All Solutions: type, version, maturity, owner, SDE, factory, feature list
Solution Models Registry	All Blueprints, Designs, and Prototypes; linked to parent Solutions
Vendor Registry	All approved Vendors: parts catalog, qualification status, SLA references
Tool Registry	All Tools: version, type, license, verification hash, toolset membership
Factory Registry	All Solution Factories: SDE network, governance config, produced Solutions
Artifact Registry	All build artifacts: binary, version, attestation record, storage location

SECTION 12

Solution Value Chain

The Solution Value Chain describes the end-to-end sequence of activities, inputs, and outputs that transforms a Solution concept into a delivered, operational Solution. qala makes the value chain explicit, traceable, and governable.

12.1 Value Chain Stages

Stage	Key Activities	Output
Ideation	Problem definition, goal setting, feasibility assessment	Solution Charter, value proposition
Design	Blueprint authoring, prototype creation, design review	Approved Solution Model
Build	Hermetic build, unit tests, CI pipeline execution	Verified build artifact
Test	Full test suite execution, defect resolution, security scan	Test-qualified artifact
Release	CM board review, version tagging, change log, sign-off	CM-approved release
Deploy	Deployment workflow execution to target environment	Running Solution instance
Operate	Monitoring, observability, incident response, patching	Stable operational Solution
Retire	Deprecation notice, migration path, archival	Archived Solution record

SECTION 13

Governance, Standards & Policies

Governance in qala ensures that all Solutions, Factories, and Environments operate within defined quality, security, and compliance boundaries. Governance is enforced through policies, standards, templates, playbooks, and lifecycle gate criteria.

13.1 Governance Instruments

Instrument	Description
Policy	A declarative rule that constrains behavior at the platform, factory, or solution level (e.g., "all CM solutions must have passing security scans")
Standard	A defined approach or specification that Solutions must conform to (e.g., API versioning standard, naming convention)
Template	A pre-approved scaffold for common Solution types, SDE configurations, or workflow definitions
Playbook	A step-by-step guide for executing a defined process (e.g., Incident Response Playbook, Release Playbook)
Gate Criteria	The specific conditions that must be satisfied for a Solution to advance from one maturity stage to the next
Audit Log	Immutable, tamper-evident record of all actions taken on Solutions, SDEs, and Factories
Ownership Record	Documented assignment of accountability for every Solution, Component, and SDE on the platform

13.2 Solution Ownership System

Every Solution on qala has an unambiguous owner. Ownership defines accountability for quality, security, and lifecycle decisions:

- Solution Owner: the individual or team accountable for the Solution's quality and roadmap
- Component Owner: the individual or team accountable for a specific Solution Component
- Factory Owner: the team responsible for operating a Solution Factory and its SDEs
- Governance Board: the group that approves CM-stage promotions and major lifecycle changes

SECTION 14

Solution Data Platform

The Solution Data Platform manages the full data lifecycle for the qala platform: metrics, events, analytics, master data, and AI-driven insights. Every data point is traceable to its origin Solution, SDE, or Factory.

14.1 Data Platform Capabilities

Capability	Description
Event Streaming	Apache Kafka-based platform-wide event bus carrying Solution, SDE, Build, Security, and AI events
Metrics Collection	Time-series metrics from all Solutions, SDEs, and Factories: CPU, memory, latency, error rates
Log Aggregation	Centralized, indexed log collection from all platform services and Solution environments
Data Warehouse	Columnar analytics store for historical reporting, trend analysis, and AI model training
Master Data Management	Authoritative reference data for Solution types, status codes, and classification taxonomies
Analytics Dashboards	Pre-built dashboards for Solution health, factory throughput, defect density, and resource utilization
AI Inference	Real-time and batch inference endpoints for predictive capabilities throughout the platform
Data Lineage	Full provenance tracking: every data point traces to its origin event and transformation history

14.2 Event Topics

Event Topic	Published By
SOLUTION_EVENTS	Solution Registry — creation, update, maturity transitions
USER_EVENTS	User & Identity Service
SDE_EVENTS	SDE Management Service — provisioning, snapshot, rollback, lifecycle
CMS_EVENTS	Workspace & CMS Service — content create/update/delete
BUILD_EVENTS	Workflow & CI/CD Service — pipeline runs, build outcomes
ARTIFACT_EVENTS	Artifact & Package Management — uploads, downloads, deletions

DATA_EVENTS	Data Platform Service — pipeline runs, anomalies
SECURITY_EVENTS	Security & SEM Service — threats, policy violations, scans
NOTIFICATIONS	Notifications Service
AI_RECOMMENDATIONS	AI Agents Service

SECTION 15

AI Integration

AI capabilities are embedded throughout the qala Solution System — not as an add-on layer, but as an integral part of every major subsystem. The AI Agents service provides intelligent hooks across the platform.

15.1 AI Capability Map

AI Capability	Subsystem	Description
SDE Optimization	SDE Management	Recommends configuration improvements for performance and cost efficiency
Pipeline Bottleneck Detection	CI/CD	Identifies slow stages and recommends parallelization or caching
Predictive Defect Detection	Testbed	Flags high-risk code areas using historical defect patterns
Test Case Generation	Testbed	Synthesizes test scenarios from specifications and past failures
Self-Healing Tests	Testbed	Automatically updates fragile tests when interfaces evolve
Anomaly Detection	Data Platform	Detects unusual patterns in metrics, logs, and security events
Resource Prediction	Data Platform	Forecasts compute and capacity needs based on activity trends
Security Threat Analysis	Security/SEM	Correlates security events to identify attack patterns and risks
Prototype Feedback Analysis	Solution Model	Synthesizes prototype results into design recommendations
Vendor Advisory Monitoring	Tooling	Monitors tool ecosystem for advisories and recommended upgrades
Backup Schedule Optimization	SDE Management	Recommends backup frequency based on change velocity and risk
Content Suggestions	Workspace/CMS	Recommends documentation structure and template selection

SECTION 16

Security & Compliance

Security in qala is not a separate concern — it is embedded in the Solution lifecycle at every stage, enforced by the Security and Event Management (SEM) service and validated by automated scanning in every CI/CD pipeline.

16.1 Security Model

Control	Description
RBAC	Role-Based Access Control enforced at every API endpoint and SDE operation
MFA	Multi-factor authentication required for all user and service accounts
mTLS	Mutual TLS for all service-to-service communication within the platform
Secrets Vault	HashiCorp Vault or cloud-native KMS for all credentials, certificates, and secrets
SAST	Static Application Security Testing on every commit
DAST	Dynamic Application Security Testing on every build targeting TEST+ maturity
Build Attestation	SLSA provenance records signed for every build output; verified before deployment
Supply Chain Verification	All tool and dependency binaries verified against cryptographic hashes before use
Immutable Audit Logs	All platform actions are recorded in tamper-evident, append-only audit logs
Data Encryption	Encryption at rest and in transit mandatory for all Solution and tenant data
Threat Detection	Real-time SEM service monitors for anomalous activity, policy violations, and attacks
SDE Quarantine	Compromised or suspicious SDEs can be automatically isolated by the SEM service

SECTION 17

API Surface

qala exposes a versioned REST API surface. All endpoints route through the central API Gateway which enforces authentication, authorization, rate limiting, and observability.

17.1 Solution System Endpoints

Endpoint	Method(s)	Description
/solutions	GET, POST	List all Solutions or create a new Solution
/solutions/{id}	GET, PATCH, DELETE	Retrieve, update, or delete a Solution record
/solutions/{id}/components	GET, POST	List or add Components to a Solution
/solutions/{id}/components/{cid}	GET, PATCH, DELETE	Manage a specific Component
/solutions/{id}/components/{cid}/parts	GET, POST	List or add Parts to a Component
/solutions/{id}/model	GET, PUT	Retrieve or update the Solution Model (blueprint/design/prototype)
/solutions/{id}/features	GET, POST	List or add Features to the Solution features list
/solutions/{id}/features/{fid}	GET, PATCH, DELETE	Manage a specific Feature
/solutions/{id}/maturity	GET, POST	Get maturity status or submit a maturity promotion request
/solutions/{id}/orchestration	GET, POST	View or create Workflows for a Solution
/solutions/{id}/testbed	GET	View testbed configuration and results for a Solution
/solutions/{id}/value-chain	GET	View the value chain trace for a Solution
/vendors	GET, POST	List or register Vendors
/vendors/{id}	GET, PATCH, DELETE	Manage a specific Vendor record
/tools	GET, POST	List or register Tools in the Tool Registry
/factories	GET, POST	List or create Solution Factories
/factories/{id}/sdes	GET, POST	List or create SDEs in a Factory

APPENDIX A

Glossary

Term	Definition
Solution	The central root element of qala: a typed, versioned answer to a problem or goal, composed of Components and governed by a lifecycle
SDE	Solution Development Environment — the atomic operational unit providing all tools, configuration, and resources for creating a Solution
SF	Solution Factory — a coordinated collection of networked SDEs producing Solutions in a consistent, repeatable way
SFOS	Solution Factory Operating System — the qala platform itself
CM	Configuration Management — governed control of all environment and system configuration; highest maturity stage
CI	Continuous Integration — automated build and test on every code change
CD	Continuous Deployment/Delivery — automated promotion of validated artifacts to target environments
SAST	Static Application Security Testing — code analysis without execution
DAST	Dynamic Application Security Testing — testing against a running application
RBAC	Role-Based Access Control — access permissions determined by assigned roles
SLSA	Supply chain Levels for Software Artifacts — a security framework for build attestation and provenance
Toolchain	A linked, ordered sequence of Tools where each tool's output feeds the next
Blueprint	The formal structural specification of a Solution Model
Maturity	The lifecycle classification of a Solution: SANDBOX, DEV, NIGHTLY, TEST, or CM
Hermetic Build	A fully isolated, reproducible build environment with all dependencies frozen
Value Chain	The end-to-end sequence of activities transforming a Solution concept into an operational Solution

APPENDIX B

Solution Maturity Gate Criteria

Maturity Promotion	Required Gate Criteria
SANDBOX → DEV	Solution record created in Registry; SDE provisioned; owner assigned
DEV → NIGHTLY	CI pipeline configured; build succeeds; basic unit tests passing
NIGHTLY → TEST	All unit and integration tests passing on main branch; SAST scan clean; no P1 defects open
TEST → CM	All test suites passing; DAST scan clean; performance benchmarks met; governance board sign-off; build attestation generated